PEMROGRAMAN MOBILE
PERTEMUAN 3 - 3 MARET 2026

NAMA : FAJAR KURNIA PUTRA

KELAS : SIB 2F

ABSEN : 09

NIM : 244107060074

TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG

JAWA TIMUR

MALANG

2026

A. Studi Kasus Perkalian

```
void main() {
  int angka = 7;

  if (angka <= 5) {
    // Format sederhana untuk angka kecil
    for (int i = 1; i <= 10; i++) {
      print('$angka x $i = ${angka * i}');
    }
  } else {
    // Format tabel untuk angka besar
    print('Tabel perkalian $angka:');
    print('------------------');

    for (int i = 1; i <= 10; i++) {
      int hasilKali = angka * i;
      String hasil = '$angka x $i = $hasilKali';

      // Tambahkan highlight untuk hasil genap
      if (hasilKali % 2 == 0) {
        print('$hasil (genap)');
      } else {
        print(hasil);
      }
    }
  }
}
```

```
Tabel perkalian 7:
------------------
7 x 1 = 7
7 x 2 = 14 (genap)
7 x 3 = 21
7 x 4 = 28 (genap)
7 x 5 = 35
7 x 6 = 42 (genap)
7 x 7 = 49
7 x 8 = 56 (genap)
7 x 9 = 63
7 x 10 = 70 (genap)

Exited.
```

Analisa: Program mulai dari fungsi main lalu menetapkan nilai angka yaitu 7. Setelah itu program mengecek apakah angka kurang dari atau sama dengan 5. Karena 7 lebih besar dari 5, program masuk ke bagian else dan menampilkan judul tabel perkalian. Selanjutnya program menjalankan perulangan dari 1 sampai 10. Pada setiap perulangan, program menghitung hasil perkalian angka dengan nilai perulangan, lalu mengecek apakah hasilnya habis dibagi 2. Jika habis dibagi 2 maka ditambahkan keterangan genap, jika tidak maka ditampilkan biasa. Proses ini berulang sampai angka 10 lalu program selesai.

B. Studi kasus BMI

```dart
// main.dart
Run | Debug | Windsurf: Refactor | Explain | Generate Function Com
void main() {
  List<Map<String, dynamic>> riwayat = [];

  // Simulasi beberapa input
  hitungBMI(170, 65, riwayat);
  hitungBMI(165, 75, riwayat);
  hitungBMI(180, 85, riwayat);

  // Tampilkan riwayat
  tampilkanRiwayat(riwayat);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void hitungBMI(
  double tinggiCm,
  double beratKg,
  List<Map<String, dynamic>> riwayat,
) {
  // Konversi tinggi dari cm ke meter
  double tinggiM = tinggiCm / 100;

  // Hitung BMI
  double bmi = beratKg / (tinggiM * tinggiM);

  // Tentukan kategori
  String kategori;

  if (bmi < 18.5) {
    kategori = "Kurus";
  } else if (bmi < 25) {
    kategori = "Normal";
  } else if (bmi < 30) {
    kategori = "Gemuk";
  } else {
    kategori = "Obesitas";
  }
```

```dart
  // Simpan ke riwayat
  riwayat.add({
    'tinggi': tinggiCm,
    'berat': beratKg,
    'bmi': bmi,
    'kategori': kategori,
  });
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
void tampilkanRiwayat(List<Map<String, dynamic>> riwayat) {
  for (int i = 0; i < riwayat.length; i++) {
    var data = riwayat[i];
    print(
      'Data ${i + 1}: Tinggi ${data['tinggi']} cm, '
      'Berat ${data['berat']} kg, '
      'BMI ${data['bmi'].toStringAsFixed(2)}, '
      'Kategori ${data['kategori']}',
    );
  }
}
```

```
Data 1: Tinggi 170.0 cm, Berat 65.0 kg, BMI 22.49, Kategori Normal
Data 2: Tinggi 165.0 cm, Berat 75.0 kg, BMI 27.55, Kategori Gemuk
Data 3: Tinggi 180.0 cm, Berat 85.0 kg, BMI 26.23, Kategori Gemuk
```

Analisa: Program mulai dari main dan membuat list riwayat kosong. Program memanggil fungsi hitungBMI beberapa kali dengan data tinggi dan berat berbeda. Di dalam hitungBMI, program mengubah tinggi ke meter lalu menghitung nilai BMI. Setelah itu program menentukan kategori berdasarkan nilai BMI menggunakan percabangan atau kondisi. Hasil perhitungan disimpan ke dalam list riwayat dalam bentuk map. Setelah semua data diproses, program memanggil tampilkanRiwayat untuk menampilkan seluruh data menggunakan perulangan.

C. Tugas

1. Commit pertama pada branch main, dimana saya membuat game sederhana berbasis percabangan dimana satu orang menebak satu angka. Jika angka yang diinput sama seperti angka yang di generate oleh sistem, maka orang tersebut terkena kutukan kesialan.

```dart
//import library
import 'dart:io';
import 'dart:math';

Run | Debug | Windsurf: Refactor | Explain | Generate Function Comment | ×
void main() {
//variabel random untuk memakai generator angka random
  var random = Random();
  //variabel sial untuk menampung angka random
  int sial = random.nextInt(10) + 1;

//Tampilkan teks pada terminal
  print('=== Unlucky Roulette ===');
  stdout.write('Pilih angka aman (1-10): ');
  //variabel tebakan untuk menampung angka yang diinput
  int tebakan = int.parse(stdin.readLineSync()!);

//Kondisi untuk mengecek tebakan
  if (tebakan == sial) {
    print('BOOM! Kamu terkena kutukan di angka $sial!');
  } else {
    print('HORE! Kamu beruntung. Kutukan ada di angka $sial.');
  }
}
```

```
=== Unlucky Roulette ===
Pilih angka aman (1-10): 2
HORE! Kamu beruntung. Kutukan ada di angka 4.
```

Analisa: Sistem generate angka random melalui library dan disimpan kedalam variabel sial dan nantinya akan dicocokkan terhadap angka yang diinputkan user pada terminal. Angka tersebut disimpan dalam variabel tebakan yang nantinya akan di compare dengan variabel sia. Jika kondisi and terpenuhi dimana angka tebakan dan angka sial adalah sama maka user terkena kutukan, namun jika tidak, user aman.

2. Commit dua pada branch multiplayer, dimana saya membuat fitur multiplayer maks 4 orang.

```dart
//import library
import 'dart:io';
import 'dart:math';

Run | Debug | Windsurf: Refactor | Explain | Generate Function Comment | X
void main() {
  // Variabel random untuk generator angka
  var random = Random();
  // Variabel untuk menampung jumlah pemain
  int? jumlahPemain;

  print('=== Unlucky Roulette: Multiplayer Edition ===');

  // Perulangan untuk memvalidasi jumlah pemain (1-4)
  do {
    stdout.write('Masukkan jumlah pemain (1-4): ');
    // Membaca input dan mengubah ke integer
    jumlahPemain = int.tryParse(stdin.readLineSync() ?? '');

    // Cek jika input kosong atau di luar jangkauan 1-4
    if (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4) {
      print('Error: Pemain minimal 1 dan maksimal 4 orang!');
    }
  } while (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4);

  // Variabel sial untuk menampung angka random 1-10
  int sial = random.nextInt(10) + 1;

  // Perulangan untuk menjalankan giliran setiap pemain
  for (int i = 1; i <= jumlahPemain; i++) {
    print('\n--- Giliran Pemain $i ---');
    stdout.write('Pilih angka aman (1-10): ');
    // Variabel tebakan untuk menampung input pemain
    int tebakan = int.parse(stdin.readLineSync()!);

    // Kondisi mengecek apakah pemain terkena angka sial
    if (tebakan == sial) {
      print('Kamu terkena kutukan di angka $sial!');
      break; // Permainan berhenti jika ada yang kena
    } else {
      print('Pemain $i aman. Kutukan bukan di angka $tebakan.');
    }
  }
}
```

```
=== Unlucky Roulette: Multiplayer Edition ===
Masukkan jumlah pemain (1-4): 4

--- Giliran Pemain 1 ---
Pilih angka aman (1-10): 1
Pemain 1 aman. Kutukan bukan di angka 1.

--- Giliran Pemain 2 ---
Pilih angka aman (1-10): 3
Pemain 2 aman. Kutukan bukan di angka 3.

--- Giliran Pemain 3 ---
Pilih angka aman (1-10): 4
Kamu terkena kutukan di angka 4!
```

Analisa: Program mulai dengan meminta jumlah pemain dan memvalidasi input agar hanya menerima angka 1 sampai 4 menggunakan do while. Setelah input valid, program membuat satu angka sial secara acak dari 1 sampai 10. Lalu program menjalankan perulangan sesuai jumlah pemain, setiap pemain mendapat giliran memilih angka dengan validasi agar tidak kosong dan tidak di luar rentang. Setelah angka valid, program membandingkan dengan angka sial. Jika sama, permainan langsung berhenti karena pemain terkena kutukan. Jika tidak, permainan lanjut ke pemain berikutnya sampai semua giliran selesai atau ada yang kalah.

3. Commit tiga pada branch multiplayer, dimana permainan akan terus berulang lebih dari 1 putaran.

```
//import library
import 'dart:io';
import 'dart:math';

Run | Debug | Windsurf: Refactor | Explain | Generate Function Comment | ×
void main() {
  // Variabel random untuk generator angka
  var random = Random();
  // Variabel untuk menampung jumlah pemain
  int? jumlahPemain;

  print('=== Unlucky Roulette: Multiplayer Edition ===');

  // Perulangan untuk memvalidasi jumlah pemain (1-4)
  do {
    stdout.write('Masukkan jumlah pemain (1-4): ');
    // Membaca input dan mengubah ke integer
    jumlahPemain = int.tryParse(stdin.readLineSync() ?? '');

    // Cek jika input kosong atau di luar jangkauan 1-4
    if (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4) {
      print('Error: Pemain minimal 1 dan maksimal 4 orang!');
    }
  } while (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4);

  // Variabel sial untuk menampung angka random 1-10
  int sial = random.nextInt(10) + 1;
  // List untuk menampung angka yang sudah dipilih
  List<int> tebakanTerpakai = [];
```

```
// Perulangan untuk menjalankan giliran setiap pemain
for (int i = 1; i <= jumlahPemain; i++) {
  int? tebakan;
  while (true) {
    print('\n--- Giliran Pemain $i ---');
    stdout.write('Pilih angka aman (1-10): ');
    tebakan = int.tryParse(stdin.readLineSync() ?? '');

    if (tebakan == null || tebakan < 1 || tebakan > 10) {
      print('Masukkan angka 1-10!');
    } else if (tebakanTerpakai.contains(tebakan)) {
      print('Angka $tebakan sudah diambil player lain! Pilih angka lain.');
    } else {
      // Simpan angka agar tidak bisa dipakai lagi
      tebakanTerpakai.add(tebakan);
      break;
    }
  }

  // Kondisi mengecek apakah pemain terkena angka sial
  if (tebakan == sial) {
    print('Kamu terkena kutukan di angka $sial!');
    break; // Permainan berhenti jika ada yang kena
  } else {
    print('Pemain $i aman. Kutukan bukan di angka $tebakan.');
  }
}
}
```

```
=== Unlucky Roulette: Multiplayer Edition ===
Masukkan jumlah pemain (1-4): 3

--- Giliran Pemain 1 ---
Pilih angka aman (1-10): 1
Pemain 1 aman. Kutukan bukan di angka 1.

--- Giliran Pemain 2 ---
Pilih angka aman (1-10): 1
Angka 1 sudah diambil player lain! Pilih angka lain.

--- Giliran Pemain 2 ---
Pilih angka aman (1-10): 2
Pemain 2 aman. Kutukan bukan di angka 2.

--- Giliran Pemain 3 ---
Pilih angka aman (1-10): 2
Angka 2 sudah diambil player lain! Pilih angka lain.

--- Giliran Pemain 3 ---
Pilih angka aman (1-10): 9
Pemain 3 aman. Kutukan bukan di angka 9.
```

Analisa: Program dimulai dengan menampilkan judul lalu meminta jumlah pemain dan memvalidasi input agar hanya menerima angka 1 sampai 4 menggunakan do while. Setelah jumlah pemain valid, program membuat satu angka sial secara acak dari 1 sampai 10 dan menyiapkan list untuk menyimpan angka yang sudah dipilih agar tidak bisa dipakai ulang. Selanjutnya program menjalankan perulangan sesuai jumlah pemain, setiap pemain mendapat giliran memilih angka dengan validasi agar input tidak kosong, berada di rentang 1 sampai 10, dan belum dipilih pemain lain. Jika input valid, angka disimpan ke dalam list lalu dibandingkan dengan angka sial. Jika sama, permainan langsung berhenti karena pemain terkena kutukan. Jika tidak sama,

permainan lanjut ke pemain berikutnya sampai semua pemain selesai atau ada yang kalah lebih dulu.

4. Commit keempat untuk menambahkan fitur difficult di branch fitur_difficult

```dart
import 'dart:io';
import 'dart:math';

Run | Debug | Windsurf: Refactor | Explain | Generate Function Comment | X
void main() {
  // Variabel random untuk generator angka
  var random = Random();
  // Variabel untuk menampung jumlah pemain dan rentang angka
  int? jumlahPemain;
  int rentangAngka;

  print('=== Unlucky Roulette: Difficulty Edition ===');

  // Menampilkan menu pilihan tingkat kesulitan
  print('Pilih Tingkat Kesulitan:');
  print('1. Mudah (1-15)\n2. Normal (1-10)\n3. Hardcore (1-5)');
  stdout.write('Masukkan pilihan (1/2/3): ');
  String inputDiff = stdin.readLineSync() ?? '2';

  // Logika percabangan untuk menentukan rentang angka berdasarkan kesulitan
  if (inputDiff == '1') {
    rentangAngka = 15;
    print('Mode Mudah dipilih. Peluang selamat lebih besar!');
  } else if (inputDiff == '3') {
    rentangAngka = 5;
    print('Mode HARDCORE dipilih. Siapkan mentalmu!');
  } else {
    rentangAngka = 10;
    print('Mode Normal dipilih.');
  }

  // Validasi jumlah pemain maksimal 4 orang
  do {
    stdout.write('\nMasukkan jumlah pemain (1-4): ');
    jumlahPemain = int.tryParse(stdin.readLineSync() ?? '');
    if (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4) {
      print('Error: Pemain minimal 1 dan maksimal 4 orang!');
    }
  } while (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4);
```

```
// Angka sial diacak berdasarkan rentangAngka yang sudah dipilih
int sial = random.nextInt(rentangAngka) + 1;
// List untuk mencatat angka yang sudah diambil pemain lain
List<int> tebakanTerpakai = [];

// Loop utama permainan berdasarkan jumlah pemain
for (int i = 1; i <= jumlahPemain; i++) {
  int? tebakan;
  // Loop validasi agar tidak memilih angka yang sama
  while (true) {
    print('\n--- Giliran Pemain $i ---');
    stdout.write('Pilih angka aman (1-$rentangAngka): ');
    tebakan = int.tryParse(stdin.readLineSync() ?? '');

    if (tebakan == null || tebakan < 1 || tebakan > rentangAngka) {
      print('Masukkan angka yang valid (1-$rentangAngka)!');
    } else if (tebakanTerpakai.contains(tebakan)) {
      print('Angka $tebakan sudah diambil player lain! Pilih angka lain.');
    } else {
      // Simpan angka ke list riwayat
      tebakanTerpakai.add(tebakan);
      break;
    }
  }

  // Pengecekan hasil tebakan
  if (tebakan == sial) {
    print('BOOM! Pemain $i terkena kutukan di angka $sial!');
    break;
  } else {
    print('Pemain $i aman. Kutukan bukan di angka $tebakan.');
  }
}
}
```

```
=== Unlucky Roulette: Difficulty Edition ===
Pilih Tingkat Kesulitan:
1. Mudah (1-15)
2. Normal (1-10)
3. Hardcore (1-5)
Masukkan pilihan (1/2/3): 3
Mode HARDCORE dipilih. Siapkan mentalmu!

Masukkan jumlah pemain (1-4): 2

--- Giliran Pemain 1 ---
Pilih angka aman (1-5): 1
Pemain 1 aman. Kutukan bukan di angka 1.

--- Giliran Pemain 2 ---
Pilih angka aman (1-5): 5
Pemain 2 aman. Kutukan bukan di angka 5.
```

Analisa; Program dimulai dengan menampilkan judul lalu meminta pemain memilih tingkat kesulitan. Berdasarkan pilihan, program menentukan rentang angka yang akan digunakan dan menampilkan informasi mode yang dipilih. Setelah itu program meminta jumlah pemain dan memvalidasi input agar hanya menerima angka 1 sampai 4 menggunakan do while. Jika valid, program mengacak satu angka sial sesuai

rentang kesulitan yang dipilih dan menyiapkan list untuk menyimpan angka yang sudah dipakai agar tidak bisa digunakan ulang. Selanjutnya program menjalankan perulangan sesuai jumlah pemain, setiap pemain mendapat giliran memilih angka dalam rentang yang sudah ditentukan dengan validasi agar tidak kosong, tidak di luar batas, dan tidak sama dengan pilihan pemain lain. Jika angka valid, program membandingkan dengan angka sial. Jika sama, permainan langsung berhenti karena pemain terkena kutukan. Jika tidak, permainan lanjut ke pemain berikutnya sampai semua pemain selesai atau ada yang kalah lebih dulu.

5. Commit kelima untuk menambahkan fitur leaderboard

```dart
// import library
import 'dart:io';
import 'dart:math';

Run | Debug | Windsurf: Refactor | Explain | Generate Function Comment |
void main() {
  // Variabel random untuk generator angka acak
  var random = Random();
  // Variabel untuk menampung jumlah pemain, rentang angka, dan angka sial
  int? jumlahPemain;
  int rentangAngka;
  int sial;

  // VARIABEL BARU: List untuk menyimpan status akhir setiap pemain untuk leaderboard
  List<String> leaderboard = [];
  // List untuk mencatat angka yang sudah diambil agar tidak dipilih ulang
  List<int> tebakanTerpakai = [];

  print('=== UNLUCKY ROULETTE: THE FINAL TOURNAMENT ===');

  // 1. FITUR DIFFICULTY: Memilih tingkat kesulitan
  print('\nPilih Tingkat Kesulitan:');
  print('1. Mudah (1-15)\n2. Normal (1-10)\n3. Hardcore (1-5)');
  stdout.write('Masukkan pilihan (1/2/3): ');
  String inputDiff = stdin.readLineSync() ?? '2';

  // Menentukan rentang angka berdasarkan input user
  if (inputDiff == '1') {
    rentangAngka = 15;
    print('>> Mode Mudah: Peluang selamat lebih besar.');
  } else if (inputDiff == '3') {
    rentangAngka = 5;
    print('>> Mode HARDCORE: Sangat berbahaya!');
  } else {
    rentangAngka = 10;
    print('>> Mode Normal.');
  }
```

```dart
// 2. VALIDASI PEMAIN: Memastikan jumlah pemain adalah 1-4 orang
do {
  stdout.write('\nMasukkan jumlah pemain (1-4): ');
  jumlahPemain = int.tryParse(stdin.readLineSync() ?? '');
  if (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4) {
    print('Error: Input tidak valid! Gunakan angka 1 sampai 4.');
  }
} while (jumlahPemain == null || jumlahPemain < 1 || jumlahPemain > 4);

// Mengacak angka sial sesuai dengan rentang kesulitan yang dipilih
sial = random.nextInt(rentangAngka) + 1;

// 3. LOOP UTAMA PERMAINAN
for (int i = 1; i <= jumlahPemain; i++) {
  int? tebakan;

  // Loop validasi input tebakan (Cek validasi & Duplikat)
  while (true) {
    print('\n--- Giliran Pemain $i ---');
    stdout.write('Pilih angka aman (1-$rentangAngka): ');
    tebakan = int.tryParse(stdin.readLineSync() ?? '');

    // Cek apakah input valid (angka dan sesuai rentang)
    if (tebakan == null || tebakan < 1 || tebakan > rentangAngka) {
      print('Masukkan angka yang benar (1-$rentangAngka)!');
    }
    // Cek fitur No Duplicate: apakah angka sudah pernah diambil
    else if (tebakanTerpakai.contains(tebakan)) {
      print('Angka $tebakan sudah diambil player lain! Cari angka lain.');
    }
    else {
      // Simpan angka ke list agar tidak bisa dipakai pemain selanjutnya
      tebakanTerpakai.add(tebakan);
      break; // Keluar dari loop validasi input
    }
  }
```

```
  // 4. LOGIKA PENGECEKAN & LEADERBOARD
  if (tebakan == sial) {
    print('BOOM! Pemain $i terkena kutukan di angka $sial!');
    // Tambahkan status ELIMINASI ke catatan leaderboard
    leaderboard.add('Pemain $i: TERELIMINASI 💀');

    // Jika ada pemain yang kalah, pemain setelahnya dianggap tidak sempat main
    for (int j = i + 1; j <= jumlahPemain; j++) {
      leaderboard.add('Pemain $j: Belum Sempat Main ⏩');
    }
    break; // Hentikan seluruh permainan karena sudah ada yang kalah
  } else {
    print('Pemain $i selamat! Angka $tebakan aman.');
    // Tambahkan status SELAMAT ke catatan leaderboard
    leaderboard.add('Pemain $i: SURVIVOR 🛡️');
  }
}

// 5. PENUTUP & MENAMPILKAN LEADERBOARD
print('\n' + '='*40);
print('        RINGKASAN HASIL PERMAINAN');
print('='*40);

// Melakukan perulangan untuk mencetak isi list leaderboard satu per satu
for (var baris in leaderboard) {
  print('  $baris');
}

print('----------------------------------------');
print('Angka kutukan yang harus dihindari: $sial');
print('='*40);
print('   Terima kasih sudah bertahan hidup!');
}
```

```
=== UNLUCKY ROULETTE: THE FINAL TOURNAMENT ===

Pilih Tingkat Kesulitan:
1. Mudah (1-15)
2. Normal (1-10)
3. Hardcore (1-5)
Masukkan pilihan (1/2/3): 3
>> Mode HARDCORE: Sangat berbahaya!

Masukkan jumlah pemain (1-4): 4

--- Giliran Pemain 1 ---
Pilih angka aman (1-5): 1
Pemain 1 selamat! Angka 1 aman.

--- Giliran Pemain 2 ---
Pilih angka aman (1-5): 2
Pemain 2 selamat! Angka 2 aman.

--- Giliran Pemain 3 ---
Pilih angka aman (1-5): 3
Pemain 3 selamat! Angka 3 aman.

--- Giliran Pemain 4 ---
Pilih angka aman (1-5): 4
BOOM! Pemain 4 terkena kutukan di angka 4!

========================================
        RINGKASAN HASIL PERMAINAN
========================================
  Pemain 1: SURVIVOR 🛡️
  Pemain 2: SURVIVOR 🛡️
  Pemain 3: SURVIVOR 🛡️
  Pemain 4: TERELIMINASI 💀
----------------------------------------
Angka kutukan yang harus dihindari: 4
========================================
   Terima kasih sudah bertahan hidup!
```

Analisa: Program dimulai dengan memilih tingkat kesulitan yang menentukan rentang angka, lalu meminta jumlah pemain dengan validasi agar hanya 1 sampai 4 orang. Setelah itu program mengacak satu angka sial sesuai rentang yang dipilih. Permainan berjalan dengan loop berdasarkan jumlah pemain, setiap pemain mendapat giliran memilih angka dengan validasi agar tidak kosong, tidak di luar rentang, dan tidak sama dengan pilihan pemain lain. Jika tebakan sama dengan angka sial, pemain langsung tereliminasi, status dicatat ke leaderboard, pemain setelahnya dianggap belum sempat main, dan permainan berhenti. Jika tidak sama, pemain dinyatakan selamat dan statusnya dicatat. Di akhir permainan, program menampilkan ringkasan hasil dan angka kutukan.